

小智 AI 固件修改记录:

添加 WeXCube SDK, 添加舵机 (SG90) 及氛围灯 (WS2812) 设备控制。
WeXCube SDK 通过 Uart1 连接外部 BLE 蓝牙模块与 WeXCube 小程序通信。
代码修改平台 VSCode + ESP-IDF。
使用的小智 AI 固件版本为 1.5.5。

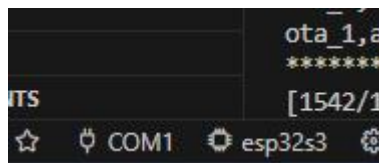
修改后的工程文件可以在

<https://gitee.com/jeremywang0102/wexcube/tree/master/Examples/example6>

或者 QQ 群 478421917 群文件中下载。

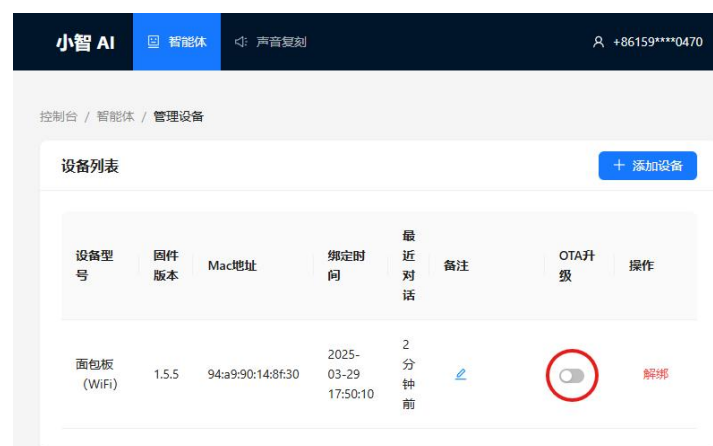
步骤:

1、选择对应的芯片类型和下载串口号

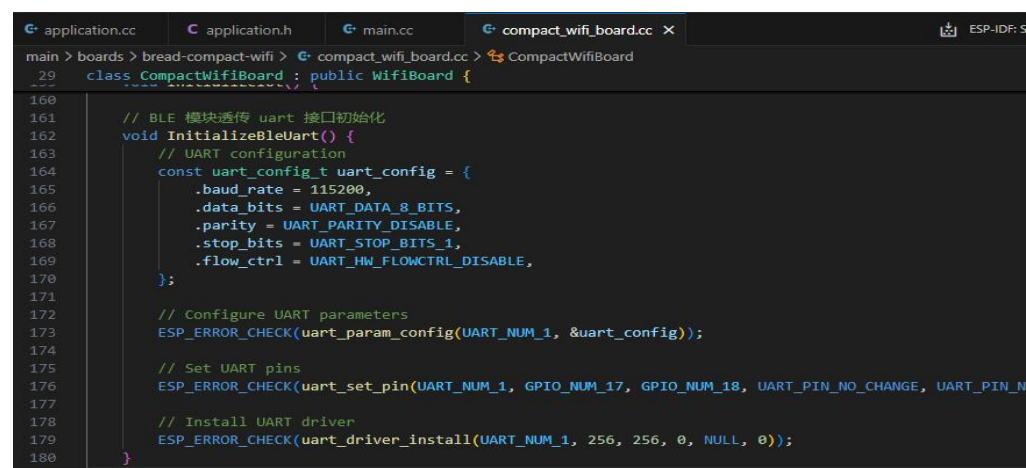


2、关闭 OTA 升级检查, 防止自己写的固件被覆盖

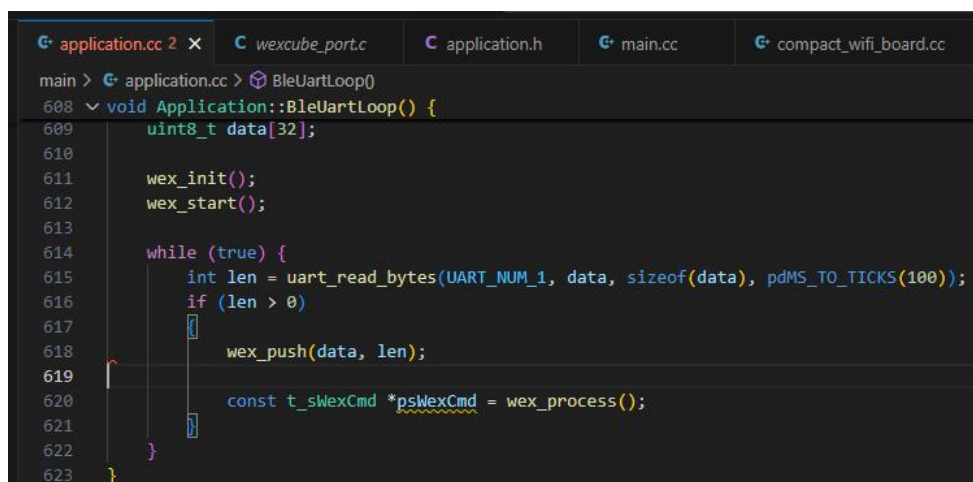
在小智 AI 后台关闭 OTA 升级



3、在 main/boards/bread-compact-wifi/compact_wifi_board.cc 添加 BLE 模块透传 uart 接口初始化代码

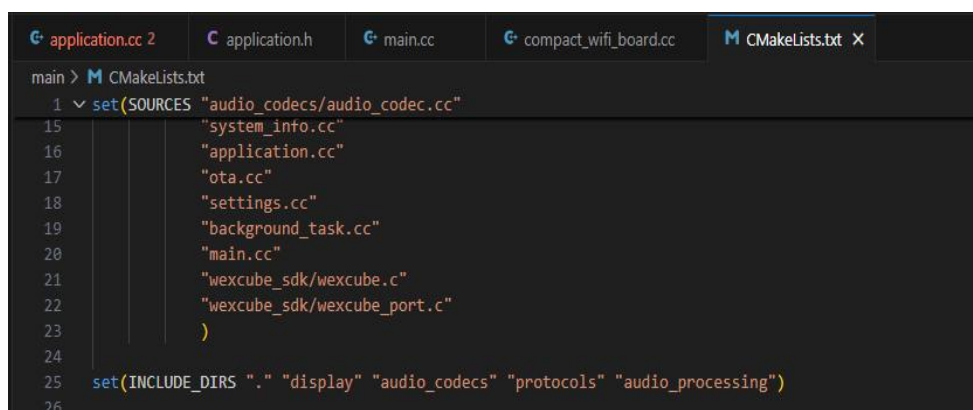


4、在 main/application.cc 中添加 BleUartLoop 线程处理 WeXCube 数据



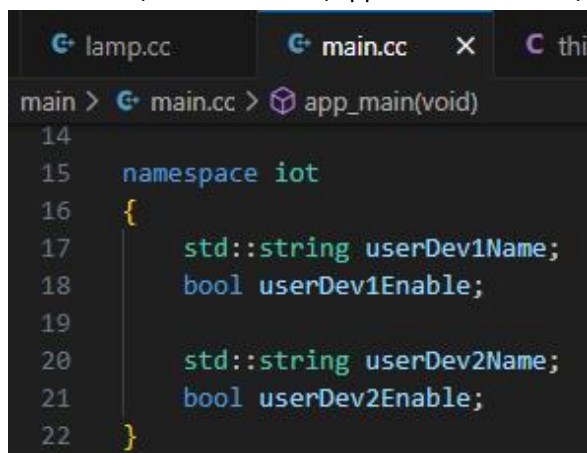
```
main > application.cc 2 x wexcube_port.c application.h main.cc compact_wifi_board.cc
main > application.cc > BleUartLoop()
608 void Application::BleUartLoop() {
609     uint8_t data[32];
610
611     wex_init();
612     wex_start();
613
614     while (true) {
615         int len = uart_read_bytes(UART_NUM_1, data, sizeof(data), pdMS_TO_TICKS(100));
616         if (len > 0)
617             wex_push(data, len);
618
619         const t_sWexCmd *psWexCmd = wex_process();
620     }
621 }
622
623 }
```

5、在 main/CMakeLists.txt 添加 wexcube SDK 的 c 文件



```
main > M CMakeLists.txt
1 set(SOURCES "audio_codecs/audio_codec.cc"
15     "system_info.cc"
16     "application.cc"
17     "ota.cc"
18     "settings.cc"
19     "background_task.cc"
20     "main.cc"
21     "wexcube_sdk/wexcube.c"
22     "wexcube_sdk/wexcube_port.c"
23 )
24
25 set(INCLUDE_DIRS "." "display" "audio_codecs" "protocols" "audio_processing")
26
```

6、在 main/main.cc、main/application.cc、main/iot/thing.h 三个文件添加设备变量声明



```
main > lamp.cc main.cc x thi
main > main.cc > app_main(void)
14
15 namespace iot
16 {
17     std::string userDev1Name;
18     bool userDev1Enable;
19
20     std::string userDev2Name;
21     bool userDev2Enable;
22 }
```

```

application.cc X wexcube.c lamp.cc
main > application.cc > Application()
41
42 namespace iot
43 {
44     extern std::string userDev1Name;
45     extern bool userDev1Enable;
46
47     extern std::string userDev2Name;
48     extern bool userDev2Enable;
49 }
50

```

```

application.cc thing.h X wexcube.c
main > iot > thing.h > {} iot > Property > num
1 #ifndef THING_H
10
11 namespace iot {
12
13     extern std::string userDev1Name;
14     extern bool userDev1Enable;
15
16     extern std::string userDev2Name;
17     extern bool userDev2Enable;
18

```

7、在 main/boards/bread-compact-wifi/compact_wifi_board.cc 添加舵机初始化

```

config.h compact_wifi_board.cc X application.h protocol.h audio_c
main > boards > bread-compact-wifi > compact_wifi_board.cc > CompactWifiBoard > Init
33 class CompactWifiBoard : public WifiBoard {
186 void InitializeServo()
187 {
188     ESP_LOGI(TAG, "Initializing servo...");
189
190     // 配置 GPIO 为输出模式
191     gpio_config_t io_conf = {};
192     io_conf.intr_type = GPIO_INTR_DISABLE;
193     io_conf.mode = GPIO_MODE_OUTPUT;
194     io_conf.pin_bit_mask = (1ULL << GPIO_NUM_14);
195     io_conf.pull_down_en = GPIO_PULLDOWN_DISABLE;
196     io_conf.pull_up_en = GPIO_PULLUP_DISABLE;
197     esp_err_t ret = gpio_config(&io_conf);
198     if (ret != ESP_OK)
199     {
200         ESP_LOGE(TAG, "Failed to configure GPIO: %s", esp_err_to_str(ret));
201         return;
202     }
203     ESP_LOGI(TAG, "GPIO configured successfully");
204
205     // 配置 LEDC 定时器
206     ledc_timer_config_t ledc_timer = {
207         .speed_mode = LEDC_LOW_SPEED_MODE,
208         .duty_resolution = LEDC_TIMER_14_BIT, // 使用 14 位分辨率

```

8、在 main/iot/things 添加舵机类文件 servo.c

```
servo.cc x application.cc wexcube.c rgb.cc wexcube_ctrl_id.h board.h thing.h
main > iot > things > servo.cc > ...
1  #include "iot/thing.h"
2
3
4
5
6
7
8  #define TAG "Servo"
9
10 namespace iot
11 {
12     // 这里仅定义 Servo 的属性和方法, 不包含具体的实现
13     class Servo : public Thing
14     {
15     public:
16         Servo() : Thing("Servo", userDev1Name)
17         {
18             // 定义设备的属性
19             properties_.AddNumberProperty("angle", "当前角度值", [this]() -> int {
20                 return Board::GetInstance().GetServoAngle();
21             });
22
23             // 定义设备可以被远程执行的指令
24             methods_.AddMethod("SetAngle", "设置角度", ParameterList({
25                 Parameter("angle", "0到180之间的整数", kValueTypeNumber, true)
26             }), [this](const ParameterList& parameters) {
27                 Board::GetInstance().SetServoAngle(static_cast<uint8_t>(parameters["angle"].number()));
28             });
29         }
30     };
31 } // namespace iot
32
33 DECLARE_THING(Servo);
```

9、在 main/boards/bread-compact-wifi/compact_wifi_board.cc 添加氛围灯初始化

```
compact_wifi_board.cc x application.cc wexcube.c rgb.cc wexcube_ctrl_id.h board.h
main > boards > bread-compact-wifi > compact_wifi_board.cc > CompactWifiBoard > InitializeIoT()
31 class CompactWifiBoard : public WifiBoard {
253 void InitializeRGB() {
255
256     // Create RMT TX channel configuration
257     rmt_tx_channel_config_t tx_config = {
258         .gpio_num = GPIO_NUM_11, // WS2812B data pin
259         .clk_src = RMT_CLK_SRC_DEFAULT, // Default clock source
260         .resolution_hz = 10000000, // 10 MHz resolution (1 tick = 0.1us)
261         .mem_block_symbols = 64, // Memory block size (must be valid for your hardware)
262         .trans_queue_depth = 4, // Transmission queue depth
263         .flags = {
264             .invert_out = false, // Do not invert output
265             .with_dma = false, // Do not use DMA
266         },
267     };
268
269     if (!tx_channel_) {
270         ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_config, &tx_channel_));
271         ESP_ERROR_CHECK(rmt_enable(tx_channel_));
272
273         // Create the encoder
274         rmt_copy_encoder_config_t encoder_config = {};
275         ESP_ERROR_CHECK(rmt_new_copy_encoder(&encoder_config, &encoder_));
276     }
277
278     SendRGBWithRMT(0, 0, 0); // Initialize with black color
279
280     ESP_LOGI(TAG, "RMT initialized successfully");
281 }
```


10、在 main/iot/things 添加氛围灯类文件 rgb.c

```
rgb.c compact_wifi_board.cc application.cc wexcube.c wexcube_ctrl_id.h board.h thing.h lamp.c
main > iot > things > rgb.c > ...
7  #define TAG "RGB"
8
9  namespace iot {
10
11  // 这里仅定义 RGB 的属性和方法, 不包含具体的实现
12  class RGB : public Thing {
13  public:
14      RGB() : Thing("RGB", userDev2Name) {
15          // 定义设备的属性
16          properties_.AddNumberProperty("color", "当前颜色", [this]() -> int {
17              return Board::GetInstance().GetRGBColor();
18          });
19
20          // 定义设备可以被远程执行的指令
21          methods_.AddMethod("SetColor", "设置颜色", ParameterList({
22              Parameter("color", "由三个字节组成第一个字节为红色第二个字节为绿色第三个字节为蓝色, 格式为0xRRGGBB", kValueTypeNumber, true)
23          }), [this](const ParameterList& parameters) {
24              Board::GetInstance().SetRGBColor(static_cast<int32_t>(parameters["color"].number()));
25          });
26      }
27  };
28
29  } // namespace iot
30
31  DECLARE_THING(RGB);
```

11、在 main/boards/bread-compact-wifi/compact_wifi_board.cc 中注册舵机与氛围灯设备

```
compact_wifi_board.cc rgb.c application.cc wexcube.c wexcube_ctrl_id.h
main > boards > bread-compact-wifi > compact_wifi_board.cc > CompactWifiBoard > InitializeBleUart()
31  class CompactWifiBoard : public WifiBoard {
165
166  // 物联网初始化, 添加对 AI 可见设备
167  void InitializeIot() {
168      auto& thing_manager = iot::ThingManager::GetInstance();
169      thing_manager.AddThing(iot::CreateThing("Speaker"));
170      //thing_manager.AddThing(iot::CreateThing("Lamp"));
171
172      if (iot::userDev1Enable) thing_manager.AddThing(iot::CreateThing("Servo"));
173      if (iot::userDev2Enable) thing_manager.AddThing(iot::CreateThing("RGB"));
174  }
```

没有注册添加则小智 AI 不能识别到该外部设备。

小智 AI 控制 IOT 设备机制:

servo.c、rgb.c 分别为舵机与氛围灯类文件, 该文件提供了两个接口, 一个是 `AddNumberProperty`, 用于传递当前设备状态, 另一个是 `AddMethod`, 用于控制设备状态。当你询问小智 AI 设备状态时, 他会自动调用相应设备的 `AddNumberProperty` 回调函数来获取设备状态。当你让小智 AI 控制设备时, 他会自动调用相应设备的 `AddMethod` 回调函数来设置设备状态。

有意思的地方是小智 AI 不像语音识别设备那样死板, 他会理解分析你的话给出恰当的控制。语音控制需要提前设定语音命令, 而小智 AI 不需要, 他是基于大模型分析的。

在控制舵机中, 我只告诉他需要传递 0 到 180 的值就行, 他就能理解后续的指令。

```
methods_.AddMethod("SetAngle", "设置角度", ParameterList({
    Parameter("angle", "0 到 180 之间的整数", kValueTypeNumber,
true)
```

```

    }), [this](const ParameterList& parameters) {
        Board::GetInstance().SetServoAngle(static_cast<uint8_t>(
parameters["angle"].number()));
    });

```

比如我跟他说让舵机站起来，他就会给舵机发送一个角度值。你甚至还能让它摇摆跳舞。

在氛围灯中，我也只是描述了一下数据格式，他就控制灯光。

```

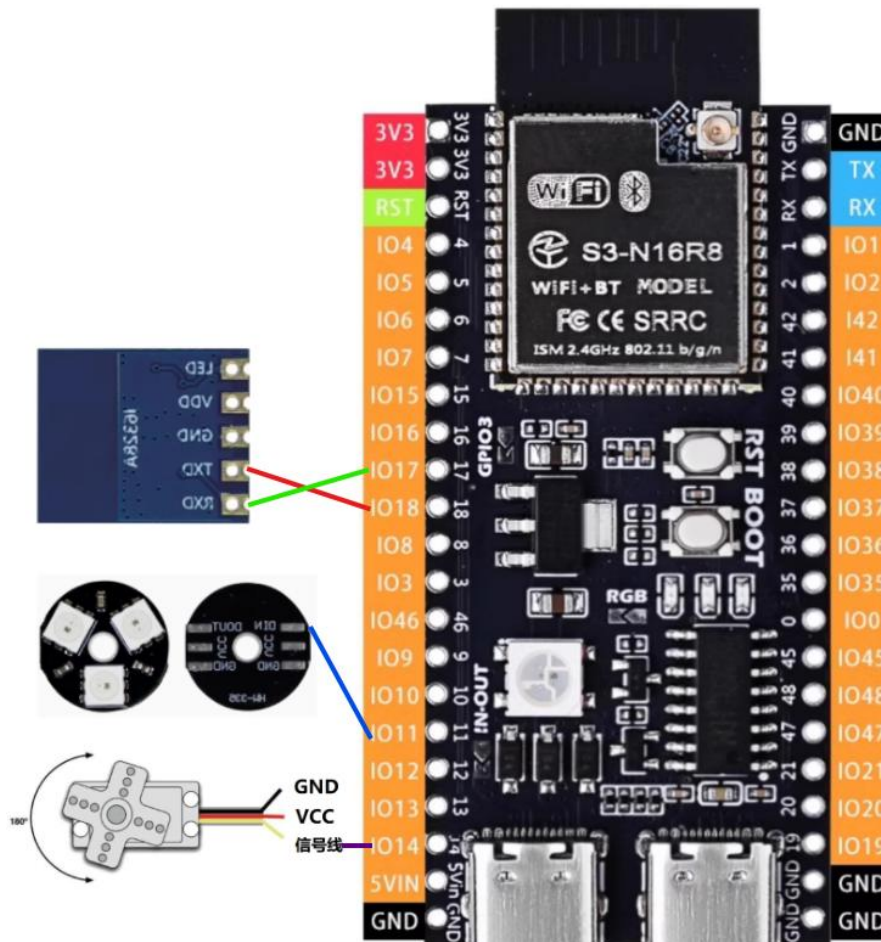
methods_.AddMethod("SetColor", "设置颜色", ParameterList({
    Parameter("color", "由三个字节组成第一个字节为红色第二个字节为绿
色第三个字节为蓝色，格式为 0xRRGGBB", kValueTypeNumber, true)
}), [this](const ParameterList& parameters) {
    Board::GetInstance().SetRGBColor(static_cast<int32_t>(parame
ters["color"].number()));
});

```

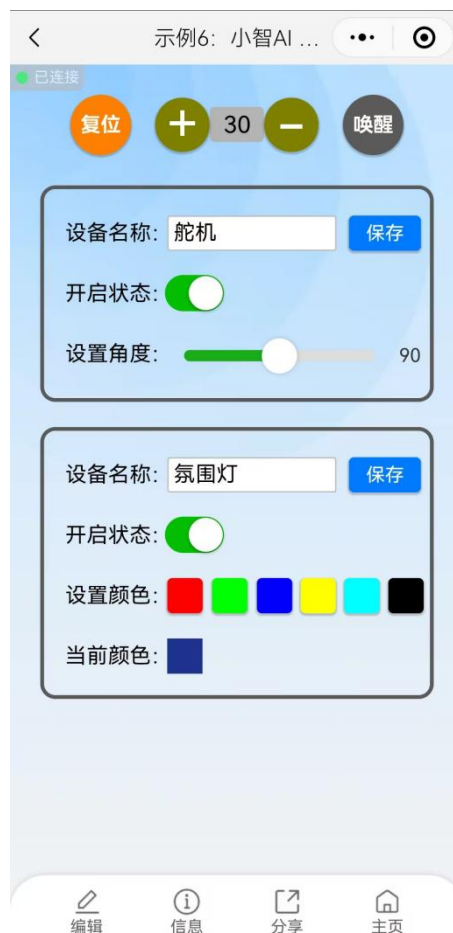
他能根据要求发生 rgb 三个字节的数据，比如我要他变成红色，他就会发送 0x00FF0000。还能让它闪烁，而我只是做了 ws2812 的驱动，灯光的变化和颜色我也没有预设。

当然小智 AI 控制 IOT 还是有些不足，比如要它控制氛围灯显示黄色，他有时可能会显示错误，在小智 AI 后台切换不同的大模型效果也不一样。

硬件连接示意图：



小程序控制页面：



页面上部分为小智 AI 的控制：设备复位、声音控制、语音对话控制。如果复位失败可以使用设备的复位按键。

页面第一个框为舵机配置，可以修改设备名称，但尽量填写与舵机相关的名称，比如机械臂之类的，填写好就可以点击保存，然后复位小智 AI，这样小智 AI 就能识别新的名称了。可以问他：“你有什么设备可以控制？”，如果他回答的还是之前的名称可以上小智 AI 后台把上面的记忆体清空，保存后再复位小智。当开启状态未打开，则小智 AI 不能识别该外部设备。

页面第二个框为氛围灯配置，操作与第一个框类似，填写要与氛围灯相关的名称。

小程序页面二维码：

